

Fundamentals of Fog and Edge Computing

BCSE313L

WINTER 2024 - 2025

Module 4 - Optimization Problems in Fog and Edge Computing

Optimization Problem - Definition

- An optimization problem is essentially about finding the best solution from a set of possible solutions.

Four Main Components in Optimization Problem

Variables

Domains

Constraints

Objective
Function

Optimization Problem - Definition

Variables

- These are the things we **can change or adjust to find the best solution**. For example, if you're planning a trip, variables could include the time you leave, the route you take, or the mode of transportation.

Domains

- Each variable has a **range of possible values it can take**. This is called the "domain" of the variable. For instance, if one of your variables is "time of departure," its domain might be all times between 6:00 AM and 9:00 AM.

Constraints

- These are **rules or conditions that must be satisfied**. For example, you might have a constraint that says you must arrive at your destination before 5:00 PM, or you cannot exceed a certain budget for fuel costs.

Objective Function

- This is the **goal you want to achieve**, expressed as a **mathematical function**. For example, you might want to minimize travel time, minimize cost, or maximize comfort.
- The objective function **assigns a numerical value to each combination of variable values**, and your goal is **to find the combination** that gives the best value for this function.

Optimization Problem - Definition

- An optimization problem is defined by:
 - A list of **variables** $x = (x_1, \dots, x_n)$
 - The **domain** D_i for each variable x_i , which specifies the set of valid values that x_i can take.
 - A set of constraints $C_1, \dots, C_m$.
 - Each **constraint** C_j relates to certain variables and **defines the valid combinations** of their values as a subset $R_j \subseteq D_{j1} \times \dots \times D_{jk}$.
 - An objective function $f: D_1 \times \dots \times D_n \rightarrow \mathbb{R}$, which assigns a real value to each combination of variable values.

Optimization Problem - Definition

- Analogy: Let's say you're planning a road trip and want to minimize the total travel time.
- Variables :
 - x1: Time of departure
 - x2: Route choice
 - x3: Speed on highways
- Domains :
 - D1: Departure time can be between 6:00 AM and 9:00 AM.
 - D2: You can choose from three routes: Route A, Route B, or Route C.
 - D3: Speed on highways can range from 60 km/h to 100 km/h.

Optimization Problem - Definition

➤ Constraints :

- You must arrive at your destination before 5:00 PM.
- You cannot exceed the speed limit on any road.
- If you choose Route B, you must depart after 7:00 AM due to road construction.

➤ Objective Function :

- $f(x_1, x_2, x_3)$: This function calculates the total travel time based on your departure time (x_1), route (x_2), and speed (x_3).
 - Your goal is to find the combination of x_1, x_2 , and x_3 that minimizes $f(x_1, x_2, x_3)$.

Optimization Problem - Objectives

- The objective of optimization is to find the best combination of variable values that satisfies all the constraints and gives the best result according to the objective function.
- In simple terms, it means finding the "best" solution from all possible solutions.

Optimization Problem - Objectives

To Achieve Objectives ensure

**Each variable value
must be within its
allowed domain**

**All constraints
must be satisfied**

**The objective
function must be
optimized**

Optimization Problem - Objectives

- The goal is to find values $v_1, \dots, v_n$ for the variables such that:
 - Each v_i belongs to its respective domain D_i .
 - All constraints are satisfied, i.e., for any constraint C_j involving variables $x_{j1}, \dots, x_{jk}$, the tuple $(v_{j1}, \dots, v_{jk})$ must belong to R_j .
 - The objective function $f(v_1, \dots, v_n)$ is maximized.
 - For minimization problems, the goal is to minimize f , which is equivalent to maximizing $f' = -f$.

Optimization Problem - Objectives

➤ Ensure Each Variable Value is Within Its Domain :

- For x_1 , you can only choose from {6:00 AM, 7:00 AM, 8:00 AM, 9:00 AM}.
- For x_2 , you can only choose from {Route A, Route B, Route C}.
- For x_3 , you can only choose from {60 km/h, 80 km/h, 100 km/h}.

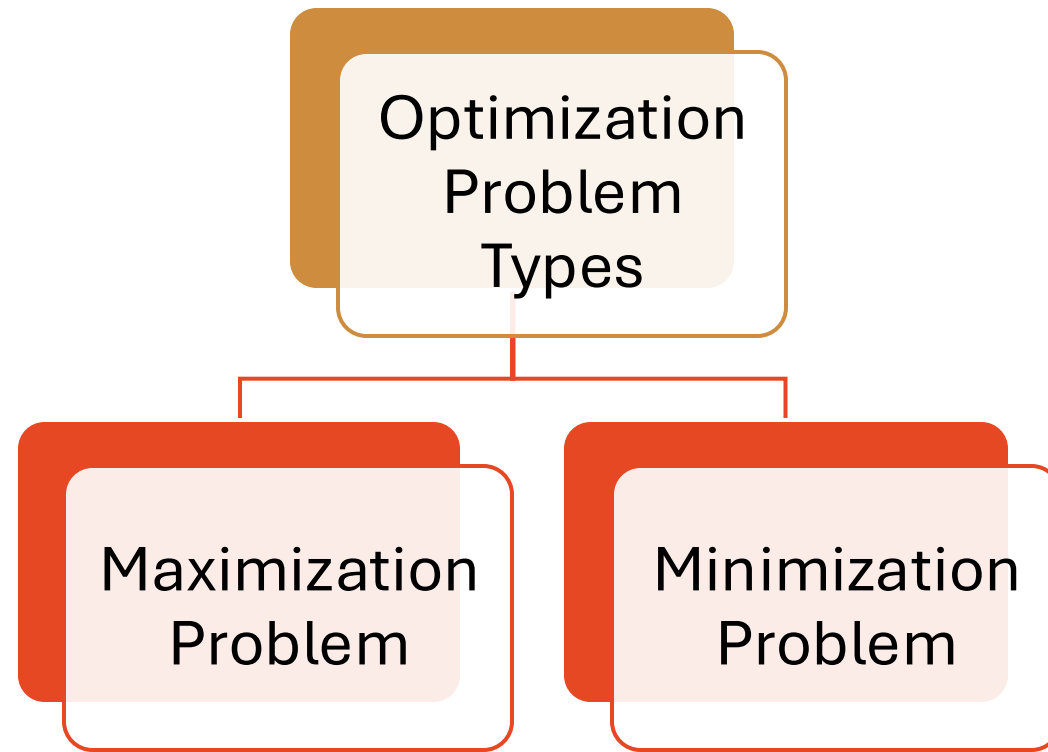
➤ Satisfy All Constraints :

- Check that the chosen combination of variables satisfies all the constraints:
 - If you choose Route B, ensure the departure time is after 7:00 AM.
 - Ensure the total travel time allows you to arrive before 5:00 PM.
 - Ensure the speed does not exceed the speed limit.

➤ Optimize the Objective Function :

- Calculate the total travel time $f(x_1, x_2, x_3)$ for each valid combination of x_1, x_2 , and x_3 .
- Find the combination that gives the **minimum travel time** .

Optimization Problem - Objectives



Maximization Problem: Find the solution that maximizes the objective function.

Minimization Problem: Find the solution that minimizes the objective function (often called a cost function).

Key Steps in Formalizing a Practical Problem

Identify Variables

- Determine what can be adjusted or changed to achieve the desired outcome. For example, in a delivery problem, variables might include the route taken, the time of departure, or the type of vehicle used.

Define Domains

- Specify the range of possible values for each variable. For instance, if one variable is "time of departure," its domain could be all times between 8:00 AM and 6:00 PM.

Set Constraints

- Define the rules or conditions that must be followed. For example, you might have constraints like "all deliveries must be completed by 5:00 PM" or "no vehicle can exceed its weight limit."

Formulate the Objective Function

- Decide what you want to optimize—minimize cost, minimize time, maximize efficiency, etc. This is expressed as a mathematical function that depends on the variables.

Key Steps in Formalizing a Practical Problem

- The process of formalization involves **abstraction**, where **non-essential details are ignored** or simplified assumptions are made.
- This helps focus on the **core aspects of the problem** but may also affect the solution's accuracy or applicability.

Formalizing a Practical Problem

- In a smart city, numerous IoT devices like traffic cameras, environmental sensors, and surveillance systems produce vast amounts of data. These devices connect to fog nodes (edge servers) that preprocess data locally before transmitting it to a central cloud server for deeper analysis. Each fog node has limited computing resources and bandwidth. The objective is to allocate tasks from IoT devices to fog nodes to minimize total latency while respecting resource constraints.

Formalizing a Practical Problem

➤ Variables

- **Task Assignment**: A decision needs to be made about which fog node will handle each task. This indicates which task is processed at which fog node
- **Latency**: Each task will have a certain amount of time (latency) associated with its processing, depending on which fog node it is assigned to.

➤ Domains

- **Task Assignment** Each task can either be processed at a specific fog node or not. There's no partial allocation – a task is either fully assigned to a node or not at all.
- **Latency** The time it takes for a fog node to process tasks will always be a positive value (or zero if it has no tasks).

Formalizing a Practical Problem

➤ Constraints

- Each task must be assigned to one and only one fog node. A task cannot be split across multiple nodes.
- The total computational demands of the tasks assigned to a fog node cannot exceed its computational capacity.
- The total bandwidth required by the tasks assigned to a fog node cannot exceed its bandwidth capacity.
- The processing time at each fog node depends on the total computational demand of the tasks it has been assigned.

Formalizing a Practical Problem

➤ Objective Function

- The goal is to minimize the total time (latency) it takes to process all tasks.
This means we want to assign tasks to fog nodes in such a way that the overall processing time across all tasks is as small as possible.

Importance of Formalization

- Without a clear understanding of the problem's structure, it becomes difficult to:
 - Analyze the problem's complexity (e.g., how hard is it to solve?).
 - Compare the problem to other known problems (e.g., does this problem resemble a well-studied one like the Traveling Salesman Problem?).
 - Evaluate whether existing algorithms can be applied effectively.

Importance of Formalization

- **Precise Reasoning** : A formal definition allows us to reason precisely about the problem. For example, we can determine if the problem is computationally easy or hard to solve.
- **Reusability** : By comparing the problem to similar ones, we might discover that an existing algorithm can be adapted to solve our problem more efficiently.
- **Clarity** : A well-defined problem ensures that everyone working on it understands the goals, constraints, and variables clearly.

Multi-objective Optimization

- In many real-world problems, we don't just have one goal to optimize; instead, we **have multiple objectives that often conflict with each other**.
- For example, when **buying a car**, you might want to **minimize cost** while **maximizing fuel efficiency and comfort**.
- These objectives can pull in different directions—**cheap cars might not be very fuel-efficient or comfortable**, while the most **fuel-efficient cars might be expensive**.
- Since it's usually **impossible to find a single solution that optimizes all objectives at once**, we use special techniques to handle multi-objective optimization problems. three common approaches: **adding lower bounds** , **scalarizing** , and finding **Pareto-optimal solutions**

Multi-objective Optimization - Adding Lower Bounds

- In this approach, you set minimum acceptable thresholds (lower bounds) for all but one objective and then focus on optimizing the remaining objective.
 - Suppose you decide:
 - Minimum fuel efficiency = 30 MPG
 - Minimum comfort rating = 7
 - Now, you want to minimize cost .
 - Filter out cars that don't meet the minimum thresholds
 - Among the remaining cars, choose the one with the lowest cost

Multi-objective Optimization - Scalarizing

- In scalarizing, you combine all objectives into a single objective function using a formula. Common methods include weighted sums or products.
 - Suppose you assign weights to each objective based on how important they are to you:
 - Cost: 50% importance
 - Fuel efficiency: 30% importance
 - Comfort: 20% importance
- To create a combined score for each car, you normalize the values (so they're on the same scale) and calculate a weighted sum.

Multi-objective Optimization - Finding Pareto-Optimal Solutions

- Pareto-optimality is about finding solutions where **improving one objective cannot be done without worsening another**. These solutions represent trade-offs between conflicting objectives.

The Case for Optimization in Fog Computing

- The main goal of fog computing is to improve certain quality attributes — specifically, things like
 - Latency ,
 - Execution time , and
 - Energy consumption
- These improvements are achieved by processing data closer to where it's generated, rather than sending everything to a distant cloud server.

Why Optimization is Important in Fog Computing

- **Minimizing Latency** : One of the key reasons fog computing exists is to reduce the delay (latency) in processing data. For example, in applications like self-driving cars or real-time health monitoring, even a small delay can be critical. By processing data locally (or near the source), fog computing reduces the time it takes for information to travel to a remote cloud server and back.
- **Improving Compute Capabilities for End Devices** : Fog computing allows devices with limited processing power (like smartphones or IoT sensors) to offload heavy computational tasks to nearby fog nodes. This means these devices can perform complex tasks faster without draining their batteries too quickly.
- **Balancing Multiple Objectives** : While reducing latency and improving compute capabilities are important, there are other factors to consider, such as energy consumption , cost , and resource utilization . These objectives often conflict with each other. For example:
 - Reducing latency might require more energy.
 - Improving compute capabilities might increase costs.
 - Using more resources at one fog node might overload it, affecting performance.

Why Optimization is Important in Fog Computing

Minimizing
Latency

Why it
matters

Many applications (e.g., IoT, autonomous vehicles, augmented reality) require real-time responses , which cloud computing often cannot provide due to high network delays.

Fog nodes, located closer to end devices, reduce the physical distance data must travel, thereby minimizing latency.

Optimization
goal

Reduce the time taken for data to travel between devices and computational resources

Why Optimization is Important in Fog Computing

Execution
Time

Why it
matters

Compute-intensive tasks (e.g., machine learning inference, video processing) need to be executed quickly to meet application deadlines.

Fog computing enables faster task execution by leveraging nearby computational resources.

Optimization
goal

Minimize the time required to process and return results for offloaded tasks.

Why Optimization is Important in Fog Computing

Energy
Consumption

Why it
matters

End devices (e.g., smartphones, IoT sensors) often have limited battery life.

Offloading tasks to fog nodes reduces the energy consumed by the device itself, as the fog node handles the heavy computations.

Optimization
goal

Minimize the energy consumption of end devices while ensuring efficient task execution.

Why Optimization is Important in Fog Computing

➤ Interactions and Conflicts Among Objectives

➤ **Nontrivial interactions :**

➤ **Optimization objectives** in fog computing are often interdependent and may conflict with one another. For example:

- **Reducing latency** might require assigning tasks to nearby fog nodes, but these nodes may already be heavily loaded, **increasing execution time** .
- **Minimizing energy consumption** on end devices might lead to **higher latency** if tasks are offloaded to distant fog nodes or the cloud.

➤ **Trade-offs :**

- Achieving optimal performance requires balancing multiple objectives, such as latency, execution time, and energy consumption.
- Multi-objective optimization techniques (e.g., Pareto optimality, weighted sums) are often needed to find solutions that balance these trade-offs.

Relevance of Optimization in Fog Computing

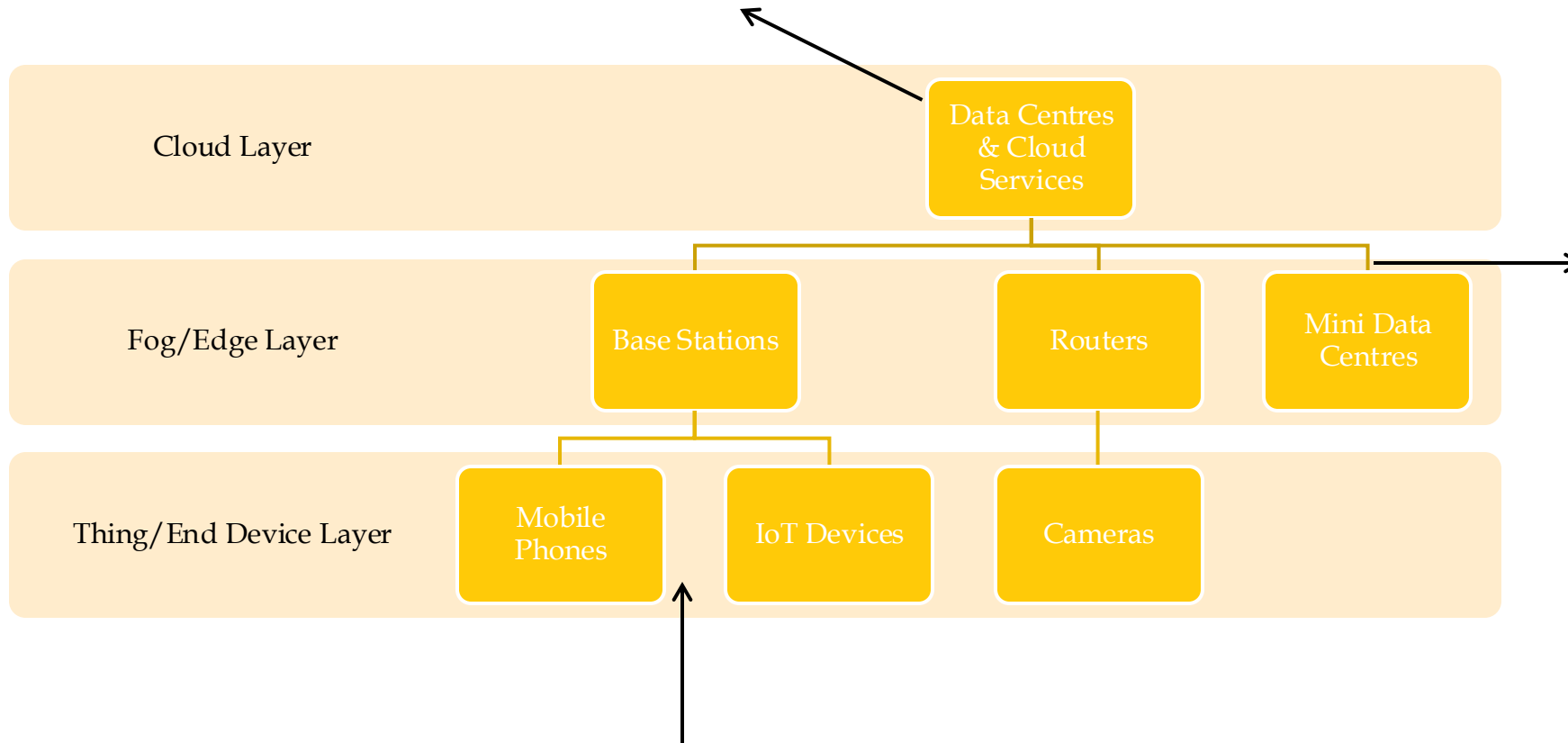
- Beyond latency, execution time, and energy consumption, other optimization objectives are also relevant:
 - **Resource utilization** : Efficiently using the computational and storage resources of fog nodes.
 - **Cost minimization** : Reducing operational costs (e.g., energy costs for running fog nodes).
 - **Reliability and fault tolerance** : Ensuring tasks are completed even if some fog nodes fail.
 - **Scalability** : Handling increasing numbers of devices and tasks without degrading performance.
- These objectives add complexity to the optimization problem, making systematic study and advanced optimization approaches essential.

Formal Modeling Framework for Fog Computing

- The framework provides a generic way to model the hierarchical structure of fog computing systems, capturing the relationships between resources, devices, and communication links.
- This formalization is essential for defining optimization problems in fog computing.

Formal Modeling Framework for Fog Computing

- Efficient in cost and energy due to large-scale infrastructure.
- Has the highest computational power
- Located far from end devices



- computing devices near the network edge
- Connected to both the cloud and end devices.

- Each end device connects to an edge resource for processing tasks.

Formal Modeling Framework for Fog Computing

➤ Define different sets and variables to model the system:

➤ Sets and Definitions

- **c**: Represents the **cloud** (single entity).
- **E**: Set of all **edge resources** (machines near the network edge).
- **D_e**: Set of **end devices** connected to a specific edge resource **e** ∈ **E**.
- **D**: The set of **all end devices**, i.e.,
$$D = \bigcup_{e \in E} D_e$$

This means D is the **union of all devices connected to different edge resources**.

- **R**: The set of **all resources** in the system, including cloud, edge, and end device
 - **R={c}∪E∪D**. Here, {c} is the cloud, E is the set of edge resources, and D is the set of all end devices.

Formal Modeling Framework for Fog Computing

➤ Computational Properties

Each resource $r \in R$ has:

- **Compute capacity:** $a(r) \in \mathbb{R}^+$ (measured in FLOPS or operations per second).
- **Compute speed:** $s(r) \in \mathbb{R}^+$ (how fast it processes instructions).
- **Power consumption per instruction:** $w(r) \in \mathbb{R}^+$ (extra power used per operation).

Note $\mathbb{R}^+$ is Real Value Positive Number (Decimal)

Formal Modeling Framework for Fog Computing

➤ Network Links and Communication

- Set of links between resources:

$$\mathbf{L} = \{\mathbf{ce} : \mathbf{e} \in \mathbf{E}\} \cup \{\mathbf{ed} : \mathbf{e} \in \mathbf{E}, \mathbf{d} \in \mathbf{De}\}.$$

This means:

- Links exist between the **cloud** and each **edge resource**.
- Links exist between each **edge resource** and its **connected end devices**.

Formal Modeling Framework for Fog Computing

➤ Each link $l \in L$ has:

- Latency $t(l) \in \mathbb{R}^+$ (delay in data transfer).
- Bandwidth $b(l) \in \mathbb{R}^+$ (data transfer capacity).
- Power cost per byte transferred: $w(l) \in \mathbb{R}^+$

Formal Modeling Framework for Fog Computing – Notations

E	set of edge resources
D_e	set of end devices connected to edge resource $e \in E$
R	set of all resources
$a(r)$	compute capacity of resource $r \in R$
$s(r)$	compute speed of resource $r \in R$
$w(r)$	marginal energy consumption of resource $r \in R$
L	set of all links between resources
$t(l)$	latency of link $l \in L$
$b(l)$	bandwidth of link $l \in L$

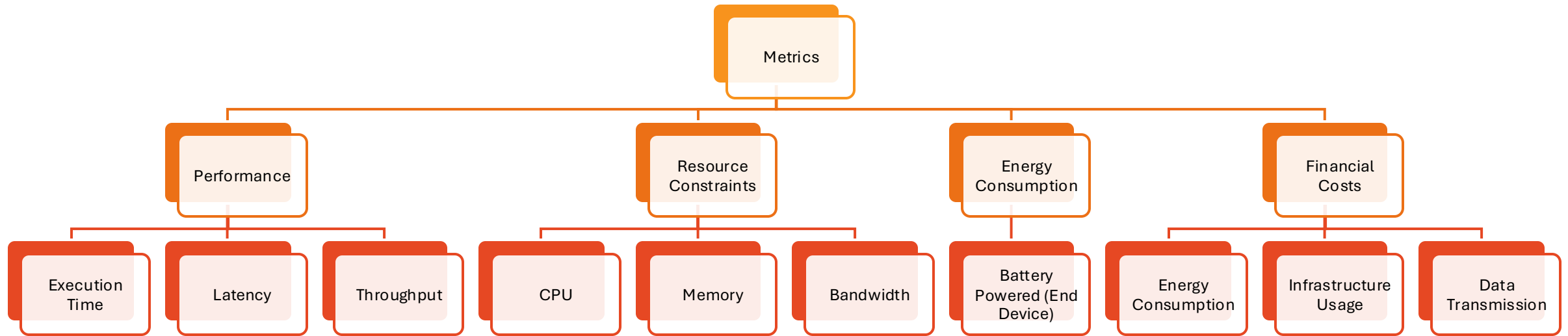
Metrics

- Metrics can be **optimization objectives or constraints**, depending on the problem variant.
 - example
 - Real-time applications: **Execution time constrained** by an upper bound, energy consumption minimized.
 - Battery-limited devices: **Energy consumption constrained**; execution time minimized.

Metrics

- In fog computing, different metrics can be used as optimization objectives (things we want to minimize or maximize) or constraints (limits we must respect).
- For example:
 - If we want a real-time application, we might constrain execution time (upper limit) and minimize energy consumption.
 - If a mobile device has a limited battery, we might constrain energy consumption and minimize execution time.

Metrics



Metrics - Performance

- Performance refers to **how fast tasks are completed**.
 - **Execution time (T_{exec})**: The total time to complete a task.
 - **Latency (T_{latency})**: Delay in data transfer between devices.
 - **Throughput (τ)**: The number of tasks completed per unit time.
- Tasks are performed using multiple resources and involve:
 1. **Computation Time (T_{comp})**: Time taken to process data on a resource.
 2. **Transfer Time (T_{trans})**: Time taken to send data between resources.
- The total execution time depends on the **critical path** (the longest sequence of dependent tasks).

If a task is split between an end device and an edge resource, the execution time is:

$$T_{\text{exec}} = T_{\text{enddevicecompute}} + T_{\text{edgetrans}} + T_{\text{edgecompute}}$$

Some tasks run **in parallel**, while others must be done **sequentially**.

The total execution time is dictated by the **slowest (critical) path**.

Metrics – Resource Usage Metrics

➤ Resource usage includes:

- **CPU usage (U_{CPU}):** How much processing power is used.
- **Memory usage (U_{mem}):** How much RAM is used.
- **Network bandwidth (U_{net}):** How much data is transferred.

➤ CPU Overbooking

- If too many tasks run on a CPU, execution slows down.
- For some applications, this is fine, but for real-time tasks, it is unacceptable.

➤ Memory Constraints

- Unlike CPU, memory cannot be overbooked easily.
- If memory runs out, applications **fail** instead of just slowing down.

Metrics – Resource Usage Metrics

➤ Network Bandwidth Constraints

- Bandwidth between **end devices and edge/cloud** is limited.
- Bandwidth usage should be minimized or constrained.

$$U_{\text{net}} \leq B_{\text{max}}$$

where:

- U_{net} = actual bandwidth usage,
- B_{max} = maximum available bandwidth.

Metrics – Energy Consumption

➤ Energy is a **scarce resource**, especially for battery-powered devices.

- **Power Consumption (P(r))**: The rate at which energy is used.
- **Energy Consumption (E(r))**: Total energy used over time.

$$E(r) = P(r) \times T_{\text{exec}}$$

- A **faster machine** may use **more power (P)** but **lower execution time (T_{exec})**, leading to lower overall energy consumption.
- It can be **more efficient** to move tasks to a faster machine.

Metrics – Energy Consumption

➤ Energy Consumption at Different Layers

- **End Devices:** Battery is limited → Energy optimization is crucial.
- **Edge Resources:** Not battery-powered → Less critical.
- **Cloud:** Energy matters due to **financial cost** (electricity is expensive).

$$E_{\text{total}} = E_{\text{end devices}} + E_{\text{edge}} + E_{\text{cloud}}$$

➤ Optimization Goal:

$$\min E_{\text{total}}$$

Metrics – Financial Cost

- **Energy consumption (E)** increases **operational cost**.
- Using **cloud or edge computing** may have:
 - **Fixed costs (C_{fixed})**: Subscription fees.
 - **Usage-based costs (C_{usage})**: Charged per unit of compute/storage.
 - **Network costs (C_{net})**: Charged per data transfer.

$$C_{\text{total}} = C_{\text{fixed}} + C_{\text{usage}} + C_{\text{net}}$$

Metrics – Formalizing Practical Problem

➤ Considering the problem given in slide 14

➤ Problem Variables

➤ We define the key variables for the problem:

➤ $D=\{d_1, d_2, \dots, d_m\}$: Set of **IoT devices** generating tasks.

➤ $F=\{f_1, f_2, \dots, f_n\}$: Set of **fog nodes** (edge servers).

➤ $T=\{T_1, T_2, \dots, T_m\}$: Set of **tasks**, where T_i is generated by device d_i .

➤ $x_{i,j} \in \{0,1\}$: **Binary decision variable**, where

➤ $x_{i,j} = 1$ if task T_i is assigned to fog node f_j ,

➤ $x_{i,j} = 0$ otherwise.

Metrics – Formalizing Practical Problem

➤ Domains (Feasible Ranges of Variables)

- Each **task** must be assigned to **exactly one fog node**:

$$\sum_{j=1}^n x_{i,j} = 1, \quad \forall i \in \{1, 2, \dots, m\}$$

- Each **fog node** has limited processing capacity C_j (CPU, memory, etc.).
- Each **fog node** has a bandwidth limit B_j .
- Each **task** has a **computational requirement** R_i (CPU cycles required).
- Each **task** has a **data transfer requirement** b_i .

Metrics – Formalizing Practical Problem

➤ Constraints

➤ We must respect the following constraints:

➤ **Task Assignment Constraint (Each Task to One Fog Node)**

➤ Every task must be processed by exactly one fog node.

$$\sum_{j=1}^n x_{i,j} = 1, \quad \forall i \in \{1, 2, \dots, m\}$$

➤ **Fog Node Compute Capacity Constraint**

➤ The total computational load assigned to each fog node must not exceed its capacity.

$$\sum_{i=1}^m R_i \cdot x_{i,j} \leq C_j, \quad \forall j \in \{1, 2, \dots, n\}$$

➤ **Fog Node Bandwidth Constraint**

➤ The total data transfer to each fog node should not exceed its bandwidth limit.

$$\sum_{i=1}^m b_i \cdot x_{i,j} \leq B_j, \quad \forall j \in \{1, 2, \dots, n\}$$

Metrics – Formalizing Practical Problem

➤ Objective Function (Minimize Total Latency)

➤ Latency includes:

1. Computation Latency $L_{\text{comp},i,j}$: Time required to execute T_i on f_j .

2. Communication Latency $L_{\text{comm},i,j}$: Time to transfer data from d_i to f_j .

➤ Total latency for a task: $L_{i,j} = L_{\text{comp},i,j} + L_{\text{comm},i,j}$

➤ Our goal is to minimize the total latency over all tasks:

$$\min \sum_{i=1}^m \sum_{j=1}^n x_{i,j} \cdot L_{i,j}$$

Quality Attributes

- **Reliability** → Ensures the system keeps working even if some parts fail.
 - **Security** → Protects data and prevents cyberattacks.
 - **Privacy** → Keeps users' personal data safe from being exposed.
- Unlike performance metrics, these quality attributes are usually addressed by designing the system properly rather than solving an optimization problem. However, they can still be included in optimization by considering trade-offs.

Quality Attributes

➤ (a) Reliability: Making Sure the System Keeps Working

- Instead of relying on a **single resource**, critical tasks can be **processed on multiple fog nodes at the same time**(**redundancy**).
- If one fog node fails, another can still complete the task.
- More redundancy **increases reliability** but also **uses more resources**.

Quality Attributes

➤ (b) Security and Privacy: Keeping Data Safe

• Choosing Trusted Resources

- Some fog nodes may be more secure than others.
- Trust scores (based on past performance or reputation) can be used to prefer safer nodes.
- Using more trusted nodes improves security but may increase cost.

• Avoiding Multi-Tenant Risks

- If different users' tasks are processed on the same fog node, it increases the risk of security attacks.
- Minimizing the number of users sharing a fog node reduces the risk but may make resource allocation less efficient.

• Reducing Data Transfers Over Networks

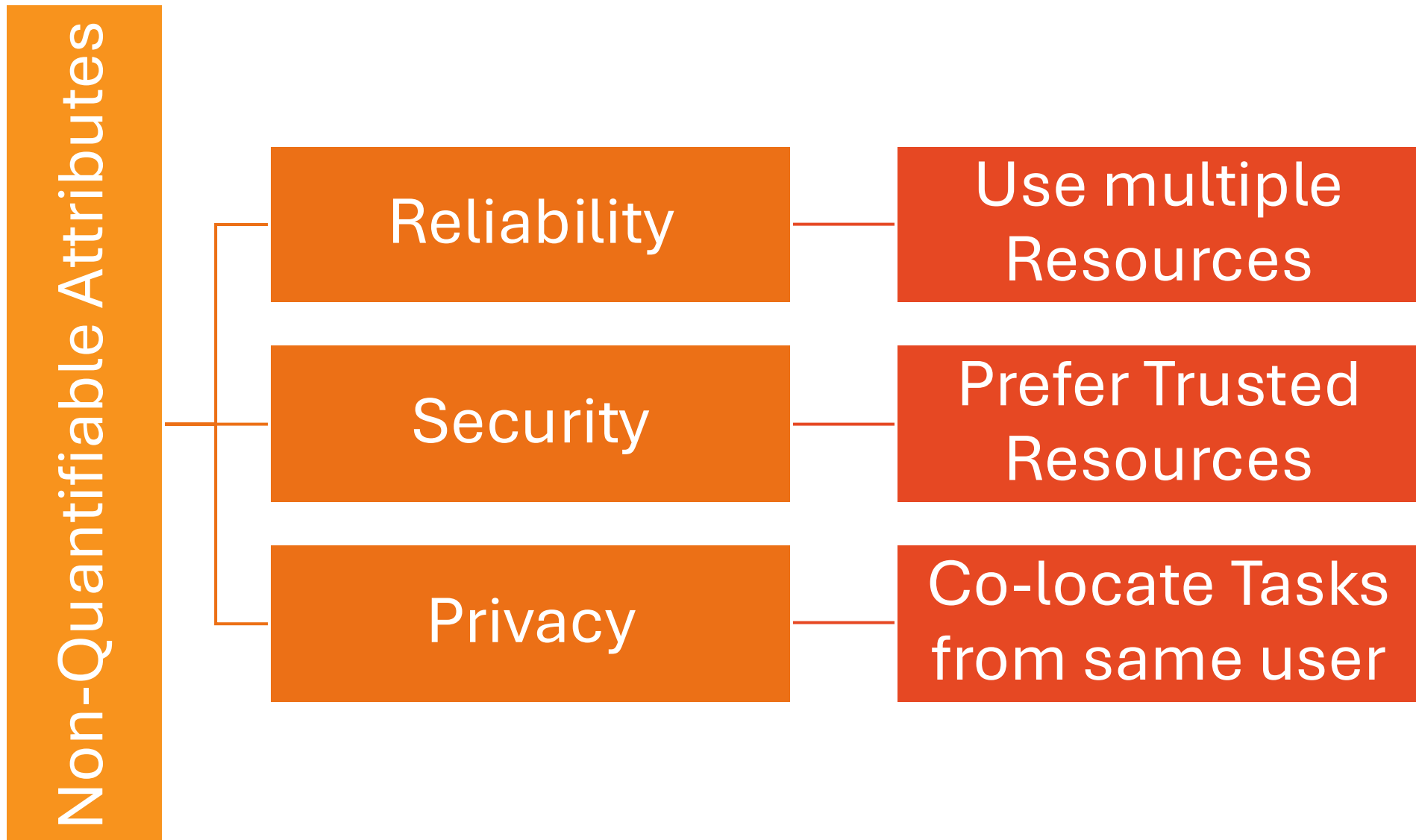
- Keeping tasks on the same fog node avoids sending sensitive data over the network.
- Less data transfer means less risk of hacking and eavesdropping.

Quality Attributes – Tradeoffs Between

Attribute	Positive Effect	Negative Trade-Off
More Redundancy (Reliability)	Increases system reliability	Uses more resources, increases cost
Using Only Trusted Nodes (Security)	Reduces cyber risks	May be more expensive
Avoiding Multi-Tenant Sharing (Privacy)	Improves security & privacy	Reduces resource efficiency

- Without considering quality attributes, a fog system may be **fast and cost-effective** but **vulnerable to failures or attacks**.
- **Optimizing these trade-offs** helps create a **secure, reliable, and efficient** fog computing system.

Quality Attributes



Optimization Opportunities along the Fog Architecture

- **Classification of Optimization Problems:**
- Optimization problems in fog computing are **classified based on the layers involved** in the three-layer fog model (cloud, edge resources, end devices).
- **Involving only one layer is typically not considered fog computing :**
 - Cloud-only optimization → Pure cloud computing.
 - End-device-only optimization → Mobile computing, IoT, or wireless sensor networks.

Optimization Opportunities along the Fog Architecture

Two-Layer Optimization Problems

Cloud and Edge Resources

Focus: Optimize metrics like overall **energy consumption** of cloud and edge resources while **respecting capacity and latency constraints**.

The **number of edge resources** can be **orders of magnitude higher** than the **number of data centers** in a distributed cloud.

Applications: Balancing load between cloud and edge resources

Edge Resources and End Devices

Focus: Collaboration between end devices and edge resources (e.g., computation offloading).

Importance: Critical due to the **limited resources** of end devices.

Single edge resource with its connected end devices → Simpler optimization problem.

Multiple edge resources with their respective end devices → More complex but allows for better load balancing across edge resources.

Applications: Task offloading, resource allocation, and load balancing

Optimization Opportunities along the Fog Architecture

➤ Three-Layer Optimization Problems

- Involvement of All Layers (Cloud, Edge Resources, End Devices)

- Challenges

 - **Computational complexity** of large-scale optimization problems with decision variables for all fog resources.

 - **Integrating diverse technical issues** into a single optimization problem (e.g., cloud concerns vs. edge vs. end-device concerns).

 - **Different stakeholders managing changes at different layers and timescales.**

- Rationale for Independent Optimization:

 - **Practical difficulties** make independent optimization of each layer more feasible.

 - **Applications:** Holistic optimization of data, code, and tasks across all layers.

Optimization Opportunities along the Fog Architecture

➤ Optimization Targets Within Each Layer

➤ Data-Related Optimization:

- Decisions about where to store and process specific pieces of data within the fog architecture.

➤ Code-Related Optimization:

- Deployment of program code across multiple resources to find the optimal placement.

➤ Task-Related Optimization:

- Splitting tasks among multiple resources to achieve optimal performance.

Optimization Opportunities along the Fog Architecture

➤ Distributed Nature of Optimization

➤ Distributed Optimization Challenges:

- Due to the distributed nature of fog computing systems, optimization may need to be performed in a **decentralized manner**.
- **Goal:** Ensure that locally optimal decisions made by autonomous resources lead to globally optimal behavior.
- **Ideal Outcome:** Achieve global optimality through collaboration among autonomous entities.

Optimization Opportunities Along the Service Life Cycle

- Fog computing, like cloud computing, involves the provision and consumption of services.
- Optimization opportunities can be categorized based on the stages of the service life cycle: **design-time , deployment-time , and run-time .**

Optimization Opportunities Along the Service Life Cycle

➤ **Design-Time Optimization**

➤ **Focus:**

- Occurs during the design phase of a fog service.
- Exact information about end devices is typically unavailable at this stage.

➤ **Key Characteristics:**

- Optimization is primarily constrained to the cloud and edge layers of the architecture.
- For end devices , optimization focuses on general types of devices rather than specific device instances (which are only known during run-time).

➤ **Applications:**

- Pre-planning resource allocation and task distribution for cloud and edge resources.

Optimization Opportunities Along the Service Life Cycle

➤ **Deployment-Time Optimization**

➤ **Focus:**

- Occurs when the service is deployed on specific resources.
- More detailed information about the resources becomes available, enabling further optimization.

➤ **Key Characteristics:**

- Exact capacities of edge resources may be known at this stage.
- Task splits between the cloud and edge resources can be (re-)optimized based on deployment-specific details.

➤ **Applications:**

- Fine-tuning resource allocation and task distribution based on actual resource capabilities.

Optimization Opportunities Along the Service Life Cycle

➤ **Run-Time Optimization**

➤ **Focus:**

- Occurs while the system is operational and actively used.
- Many critical aspects of the system become clear only during run-time.

➤ **Key Characteristics:**

➤ **Dynamic Information Availability:**

- Specific end devices and their parameters (e.g., compute capacity) are known.
- Compute tasks that end devices want to offload to edge resources are identified.

➤ **Continuous Changes:**

- System parameters (e.g., workload, network conditions) keep changing during operation.
- Requires continuous monitoring, analysis, and re-optimization to maintain system effectiveness and efficiency.

➤ **Challenges:**

- **Time Constraints:** Optimization algorithms must be fast due to limited time available during run-time.
- **Adaptation Costs:** Run-time optimization often involves adapting an existing setup rather than designing from scratch, so the costs of changes must be considered.

➤ **Applications:**

- Real-time task offloading, dynamic resource allocation, and load balancing.

Optimization Opportunities Along the Service Life Cycle

➤ Importance of Run-Time Optimization

➤ Critical Role:

- Run-time optimization is crucial because many important aspects of the system (e.g., specific devices, tasks, and conditions) are only known during operation.

➤ Consequences:

- **Fast Algorithms:** Optimization algorithms must execute quickly to adapt to dynamic conditions.
- **Adaptation Focus:** Optimization is about adapting existing setups rather than creating new ones, requiring consideration of change costs.

Toward a Taxonomy of Optimization Problems in Fog Computing

Scenario 1 :

- A video streaming service involving:
 - A central cloud data center.
 - A large number of geographically distributed edge resources (called Fog Computing Nodes, or FCNs).
 - End devices receiving streaming video.
- **Optimization Problem:**
 - Determine the optimal data rate of video streaming for each edge resource.
- **Considerations:**
 - Utility provided by different data rates at different edge resources.
 - Data center energy consumption.
 - Workload capacity of the data center.
- **Solution Approach:**
 - A distributed iterative improvement algorithm inspired by ADMM (Alternating Direction Method of Multipliers).

Toward a Taxonomy of Optimization Problems in Fog Computing

Scenario 2 Computation Offloading in Mobile Edge Computing (Sardellitti et al. [22])

A mobile edge computing (MEC) setting where:

- Mobile end devices offload compute tasks to a nearby edge resource.
- For each compute task of each end device, decisions include:
 - Whether to offload the task.
 - Which radio channel to use for communication if offloading.

➤ Optimization Problem:

- Minimize energy consumption and latency .

➤ Solution Approach:

- Single end device case:
 - Explicit closed-form solution.
- Multiple end devices with potential interference:
 - Nonconvex problem solved using an appropriate heuristic.

Toward a Taxonomy of Optimization Problems in Fog Computing

Scenario:3: Optimal Work Distribution Among Cooperating Robots (Mushunuri et al. [23])

- Robots (end devices) offload compute tasks to a server (edge resource).
- The server distributes the tasks among itself and the robots.
- Compute tasks can be split arbitrarily.
- **Optimization Problem:**
 - Find the optimal work distribution at run-time .
- **Considerations:**
 - Communication costs.
 - Battery status of devices.
 - Compute capacities of devices.
- Solution Approach:**
 - Optimization performed at run-time using an off-the-shelf nonlinear optimization package.

Toward a Taxonomy of Optimization Problems in Fog Computing

➤ **Diversity of Optimization Problems:**

➤ The three examples cover different types of optimization problems within fog computing:

- Video streaming: Focus on data rates, utility, and energy consumption.
- Computation offloading: Focus on energy, latency, and interference.
- Work distribution: Focus on communication costs, battery status, and compute capacities.

➤ **Successful Classification:**

- The presented aspects (e.g., layers involved, metrics, life cycle stages) can successfully classify approaches from the literature.
- These aspects capture the characteristics relevant for optimization in fog computing.